

Nghiên cứu phép lũy thừa nhanh trên nhóm hoán vị

Pan Zhenhao
Trường Trung học Suzhou, Jiangsu

2005

Nguồn gốc: Fast exponentiation in permutation groups: study and discussion

IOI China candidate team papers / 2005 / Pan Zhenhao / Pan Zhenhao.doc

Tóm tắt

Nhóm là một nhánh toán học lâu đời, và trong vài năm gần đây nhóm hoán vị đã có một số ứng dụng trong lập trình thi đấu. Bài viết dựa trên đặc điểm của nhóm hoán vị để đưa ra thuật toán tính lũy thừa trong thời gian tuyến tính, đồng thời dùng ví dụ để minh họa hiệu quả của thuật toán sau khi tối ưu.

Từ khóa: hoán vị; chu trình; tách; ghép.

1 Mở đầu

Nhóm hoán vị là một cấu trúc rất tốt. Trong lập trình, phần lớn các phép toán cơ bản trên nó có độ phức tạp thời gian và bộ nhớ tuyến tính, thậm chí có phép toán chỉ cần thời gian hằng số. Vì thế, nếu một bài toán có thể được trừu tượng hóa thành mô hình nhóm hoán vị, ta thường có thể giải nó khá nhẹ nhàng. Tuy nhiên, với phép lũy thừa nguyên, dường như cách trực tiếp chỉ là lặp phép nhân để thu được thuật toán $\mathcal{O}(k \cdot \text{chi phí nhân})$, hoặc dùng bình phương nhanh để thu được $\mathcal{O}(\log k \cdot \text{chi phí nhân})$. Còn với lũy thừa phân số, tức phép khai căn hoán vị, lại khó tìm được một cách hiện thực tốt.

2 Lũy thừa nguyên của nhóm hoán vị

2.1 Một phép chuyển hóa

Trong nhóm hoán vị có định lý sau. Giả sử $T^k = e$, trong đó T là một hoán vị và e là hoán vị đơn vị, tức ánh xạ mọi phần tử về chính nó. Khi đó nghiệm nguyên dương nhỏ nhất của k là bội chung nhỏ nhất của độ dài tất cả các chu trình trong phân tích chu trình của T .

Một kết luận riêng, nhưng thường dùng hơn, là: nếu T là một chu trình và $T^k = e$, thì nghiệm nguyên dương nhỏ nhất của k chính là độ dài của chu trình T .

Ta biết rằng hoán vị đơn vị là tích của nhiều chu trình chỉ chứa một phần tử. Nói cách khác, một chu trình độ dài l , sau khi lấy lũy thừa bậc l , sẽ bị tách hoàn toàn thành các chu trình đơn. Vì sao lại như vậy?

Hãy làm một thử nghiệm. Trong các ví dụ dưới đây, hoán vị đều được viết bằng cách nối các chu trình. Đặt $n = 6$, lấy

$$T = (1\ 3\ 5\ 2\ 4\ 6).$$

Khi bình phương, ta được

$$T^2 = (1\ 5\ 4)(3\ 2\ 6).$$

Chu trình đã tách thành hai phần, và hai phần ấy đúng là các phần tử ở vị trí lẻ và vị trí chẵn trong chu trình ban đầu.

Tiếp tục xét

$$T^3 = (1\ 2)(3\ 4)(5\ 6).$$

Đúng như dự đoán, nó tách thành ba phần; mỗi phần gồm các phần tử có vị trí trong chu trình ban đầu cùng dư modulo 3.

Với

$$T^4 = (1\ 4\ 5)(3\ 6\ 2),$$

khác với trường hợp trước, chu trình chỉ tách thành hai phần. Mỗi phần nhìn có vẻ lộn xộn, nhưng thực ra vẫn tương ứng với hai lớp vị trí lẻ và chẵn.

Cuối cùng,

$$T^5 = (1\ 6\ 4\ 2\ 5\ 3).$$

Lần này chu trình không tách ra, chỉ đổi thứ tự đi qua các phần tử.

Điểm chung giữa các hiện tượng trên là gì? Chính là $\gcd(k, l)$. Vì $\gcd(6, 6) = 6$, chu trình tách hoàn toàn. Tương tự, $\gcd(2, 6) = 2$, $\gcd(3, 6) = 3$, $\gcd(4, 6) = 2$, và $\gcd(5, 6) = 1$ lần lượt khớp với các kết quả thử nghiệm ở trên.

Sau nhiều thử nghiệm, ta thu được ba kết luận.

1. Nếu chu trình T có độ dài l , và l là bội của k , thì T^k là tích của k chu trình. Mỗi chu trình gồm các phần tử của T có chỉ số cùng dư $0, 1, 2, \dots$ modulo k , nối theo đúng thứ tự.
2. Nếu chu trình T có độ dài l và $\gcd(l, k) = 1$, thì T^k vẫn là một chu trình, nhưng chu trình này không nhất thiết giống T .
3. Nếu chu trình T có độ dài l , thì T^k là tích của $\gcd(l, k)$ chu trình. Mỗi chu trình gồm các phần tử của T có chỉ số cùng dư $0, 1, 2, \dots$ modulo $\gcd(l, k)$, nối theo đúng thứ tự.

Kết luận thứ ba chỉ là kết quả của việc tách k thành $\gcd(l, k) \cdot k / \gcd(l, k)$, rồi áp dụng hai kết luận đầu. Nếu các kết luận này đúng, ta chỉ cần xác định chu trình được nhắc tới trong kết luận thứ hai là đã có thể xử lý lũy thừa nguyên tùy ý của một chu trình bất kỳ trong thời gian $\mathcal{O}(n)$.

2.2 Khi độ dài chu trình nguyên tố cùng nhau với số mũ

Như ở trên, ta tiếp tục làm vài thử nghiệm. Cho

$$T = (1\ 2\ 5\ 3\ 4).$$

Khi đó

$$T^2 = (1\ 5\ 4\ 2\ 3).$$

Nếu lấy riêng các phần tử ở vị trí lẻ và chẵn của T , ta được hai dãy $(1, 5, 4)$ và $(2, 3)$. Nối hai dãy ấy lại với nhau thì đúng bằng chu trình của T^2 .

Thử thêm một lần:

$$T^3 = (1\ 3\ 2\ 4\ 5).$$

Tương tự, lấy các phần tử ở các lớp chỉ số $1, 2, 0$ modulo 3, ta được $(1, 3)$, $(2, 4)$, và (5) ; nối lại chính là chu trình mới cần tìm.

Kết quả thử nghiệm này có thể viết thành định lý sau. Giả sử

$$T = (a_0 \ a_1 \ \dots \ a_{l-1}), \quad \gcd(l, k) = 1.$$

Khi đó

$$T^k = (a_0 \ a_k \ a_{2k} \ \dots \ a_{(l-1)k}),$$

trong đó mọi chỉ số của a đều được lấy modulo l . Nói cách khác, trong chu trình ban đầu ta liên tục tiến k bước; vì k nguyên tố cùng nhau với l , quá trình này đi qua đúng một lần mọi phần tử rồi quay về điểm xuất phát.

Cách nhìn bằng hình vẽ cũng rất trực quan. Chẳng hạn với $l = 10$ và $k = 3$, ta đặt mười phần tử của chu trình trên một vòng tròn, bắt đầu từ một phần tử bất kỳ, rồi lặp lại thao tác đi thêm ba ô và đánh dấu. Thứ tự các phần tử được đánh dấu chính là chu trình của T^3 .

Chứng minh cũng không phức tạp. Trong chu trình T , áp dụng T một lần tương đương với đi từ a_i sang a_{i+1} . Vì thế áp dụng T^k một lần tương đương với đi từ a_i sang a_{i+k} , với chỉ số lấy modulo l . Khi $\gcd(l, k) = 1$, các chỉ số

$$0, k, 2k, \dots, (l-1)k \pmod{l}$$

là một hoán vị của $0, 1, \dots, l-1$. Do đó chúng tạo thành đúng một chu trình, và chu trình ấy có dạng đã nêu ở trên.

2.3 Hiện thực thuật toán

Dựa vào định lý của mục trước và ba kết luận ở mục đầu, ta dễ dàng thu được thuật toán $\mathcal{O}(n)$ cho lũy thừa nguyên. Tuy vậy, nếu hiện thực quá sát theo các mô tả lý thuyết, hằng số có thể rất lớn; đôi khi còn chậm hơn thuật toán $\mathcal{O}(n \log k)$. Để tránh điều đó, có thể dùng thuật toán rút gọn sau:

Với mỗi chu trình trong hoán vị nguồn, duyệt từng phần tử chưa được đánh dấu trong chu trình ấy. Bắt đầu từ phần tử đó, lặp lại: đánh dấu phần tử hiện tại, đưa nó vào mảng kết quả, rồi tiến k bước trong chu trình nguồn, cho đến khi quay lại phần tử ban đầu. Các phần tử đã đưa vào mảng kết quả tạo thành một chu trình trong hoán vị đích.

Thuật toán này đúng theo định lý ở mục trước và ba kết luận đã nêu, nên không cần phân tích lại chi tiết.

Ta có thể lưu các chu trình chỉ bằng hai mảng. Mảng `data` chứa liên tiếp các phần tử của mọi chu trình theo thứ tự; mảng `point` lưu vị trí bắt đầu và độ dài của từng chu trình trong `data`. Rõ ràng cả hai mảng đều có kích thước bậc $\mathcal{O}(n)$.

Vì thế, lũy thừa nguyên của nhóm hoán vị có thể được tính bằng thuật toán có độ phức tạp thời gian và bộ nhớ đều là $\mathcal{O}(n)$.

2.4 Ví dụ tối ưu: ngày lễ khánh thành

Mô tả bài toán. Nền văn minh Gustil từng rực rỡ trên Trái Đất khoảng một tỷ năm trước; đặc biệt trong lịch pháp, toán học và thiên văn, trình độ của họ đã vượt cả thời hiện đại. Trong nhiều ngôi đền cổ, các nhà khảo cổ đều phát hiện một kiến trúc kỳ lạ gồm một dãy phòng độc lập.

Ở giữa mỗi phòng treo một đĩa quay. Mỗi đĩa được chia thành p ô, và mỗi ô ghi một số từ 1 đến n . Đĩa quay có thể xoay ngược chiều kim đồng hồ; dấu đỏ trên đĩa luôn chỉ vào ô phía trên. Đĩa quay ở các phòng khác nhau là khác nhau.

Xưởng khảo cổ CC gần đây đã giải mã thành công chữ viết của nền văn minh này, qua đó biết được công dụng của kiến trúc trên. Hóa ra sau khi một ngôi đền được xây xong, cứ cách một số năm nhất định nó sẽ tổ chức một lễ khánh thành lớn. Vì “thiên cơ không thể tiết lộ”, phía đền không nói thẳng ngày lễ, mà dùng cách gợi ý. Kiến trúc kỳ lạ ấy được xây riêng để xác định ngày lễ.

Các phòng được đánh số từ trái sang phải là $1, 2, 3, \dots, n$; đồng thời đền có n tư tế cũng được đánh số từ 1 đến n . Mỗi năm, mỗi tư tế vào một phòng để cầu nguyện. Năm xây đền, tư tế số i cầu nguyện ở phòng số i . Khi một tư tế cầu nguyện trong phòng, con số ở ô đang được dấu đỏ chỉ tới chính là số phòng mà tư tế ấy sẽ tới trong năm sau. Sau khi tư tế cầu nguyện xong, đĩa quay trong phòng xoay ngược chiều kim đồng hồ một ô. Thiết kế các đĩa bảo đảm rằng trong mỗi năm, mỗi phòng có đúng một tư tế.

Sau khi đền được xây, nếu trong một năm nào đó mọi tư tế đều có số hiệu trùng với số phòng mà họ cầu nguyện, thì năm ấy là ngày tổ chức lễ. Trên thực tế, cứ sau một số năm lại có một lễ như vậy. Là cố vấn phần mềm trưởng của xưởng khảo cổ CC, bạn cần lập trình tìm ngày tổ chức lễ đầu tiên.

Dữ liệu vào. Dòng đầu gồm hai số nguyên n, p , trong đó n là số phòng, cũng là số tư tế, còn p là số ô trên mỗi đĩa quay, với $0 < n, p \leq 200$. Tiếp theo là n dòng, mỗi dòng gồm p số nguyên mô tả các số ghi trên đĩa quay của một phòng. Số đầu tiên trên dòng là số được dấu đỏ chỉ tới khi đền vừa xây; các số còn lại được cho theo chiều kim đồng hồ.

Dữ liệu ra. In ra một dòng duy nhất, là số năm sau khi xây đền mà lễ đầu tiên được tổ chức. Nếu không bao giờ xuất hiện năm thỏa điều kiện, hoặc năm thỏa điều kiện đầu tiên vượt quá 10^9 , khi đó nền văn minh Gustil đã suy tàn, hãy in `No one knows`.

Phân tích thuật toán. Vì mỗi đĩa quay có p số, và mỗi năm mỗi tư tế ở một phòng khác nhau, ta có thể biến các đĩa quay trong các phòng thành p hoán vị độ dài n . Vị trí các tư tế trong một năm cũng tạo thành một hoán vị độ dài n .

Rõ ràng hoán vị vị trí của các tư tế ở năm thứ i bằng hoán vị vị trí năm thứ $i - 1$ nối với hoán vị được chỉ bởi các số trên đĩa quay trong năm thứ i . Bài toán là tìm năm sớm nhất y sao cho hoán vị vị trí ở năm đó là hoán vị đơn vị.

Các hoán vị do đĩa quay sinh ra có chu kỳ p . Ta liệt kê $k = y \bmod p$. Gọi

$$A = Q_p Q_{p-1} \cdots Q_1$$

là hoán vị tạo bởi một chu kỳ đủ p năm, và gọi

$$B_k = Q_k Q_{k-1} \cdots Q_1$$

là phần đầu của chu kỳ. Khi đó $y = xp + k$, và hoán vị vị trí của các tư tế có dạng

$$A^x B_k$$

theo một quy ước nối hoán vị cố định. Vì phép nối có tính kết hợp, ta có thể tính trước A và từng B_k , rồi với mỗi k chuyển bài toán thành: biết hai hoán vị A và C_k , hãy tìm số nhỏ nhất

$$x = \frac{y - k}{p}$$

sao cho

$$A^x = C_k.$$

Nếu A và C_k đều là một chu trình, thuật toán lũy thừa ở trên lập tức cho ta giá trị nhỏ nhất của x , đồng thời cho biết mọi đáp án khả thi có dạng $x + tn$. Nếu có nhiều chu trình, ta sẽ thu được một hệ các phương trình đồng dư dạng

$$x \equiv x_i \pmod{n_i}.$$

Giải hệ phương trình đồng dư tuyến tính này sẽ cho đáp án.

So với thuật toán của tác giả bài gốc của đề, điểm tối ưu hơn của cách làm này nằm ở việc thiết lập từng phương trình đồng dư. Thuật toán ban đầu duyệt từ từng phần tử trong hoán vị để tìm giá trị x của mỗi phương trình đồng dư; trường hợp xấu nhất cần $\mathcal{O}(n^2)$, trở thành nút cổ chai của toàn bộ thuật toán. Hơn nữa, các phần tử trong cùng một chu trình thật ra sinh ra cùng một phương trình đồng dư, nên cách duyệt ấy lặp lại nhiều phép tính. Thuật toán ở đây tìm hệ phương trình đồng dư trong $\mathcal{O}(n)$, đẩy nút cổ chai sang bước giải hệ trong $\mathcal{O}(n \log n)$. Vì vậy độ phức tạp thời gian của toàn bộ thuật toán là $\mathcal{O}(n \log n)$.

3 Lũy thừa phân số của nhóm hoán vị: khai căn

3.1 Khai căn một chu trình

Trường hợp độ dài chu trình nguyên tố cùng nhau với số mũ. Từ định lý ở mục 2.2, ta có thể thu được quá trình tính lũy thừa phân số khi độ dài chu trình và số mũ nguyên tố cùng nhau. Ý tưởng chỉ là đảo chiều cách xây dựng: trong chu trình đích, mỗi lần con trở lùi hoặc tiến k vị trí theo quy ước đã chọn, còn trong chu trình nguồn mỗi lần con trở đi một vị trí. Nhờ $\gcd(l, k) = 1$, ánh xạ vị trí này vẫn đi qua mọi phần tử đúng một lần.

Nói cụ thể hơn, nếu cần tìm S sao cho $S^k = T$, với

$$T = (a_0 \ a_1 \ \dots \ a_{l-1}), \quad \gcd(l, k) = 1,$$

ta sắp xếp các phần tử vào chu trình S sao cho việc đi k bước trong S tương ứng với việc đi một bước trong T . Đây chính là phép nghịch đảo của công thức tính T^k ở mục 2.2.

Trường hợp độ dài chu trình không nguyên tố cùng nhau với số mũ. Nếu độ dài chu trình và số mũ không nguyên tố cùng nhau, một chu trình đơn lẻ nhất định không khai căn được. Giả sử có thể khai căn, ta xét hai khả năng.

Nếu hoán vị đích là một chu trình, theo kết luận ở mục 2.1, lũy thừa bậc k của nó sẽ tách chính nó thành $\gcd(l, k)$ phần, rõ ràng không thể bằng chu trình nguồn đơn lẻ. Nếu hoán vị đích là tích của nhiều chu trình, lũy thừa bậc k chỉ có thể tách các chu trình mà nó chứa, chứ không thể ghép các chu trình lại với nhau. Vì vậy nó cũng không thể cho ra một chu trình nguồn đơn lẻ. Do đó, khi độ dài chu trình và số mũ không nguyên tố cùng nhau, một chu trình đơn lẻ không thể được khai căn.

3.2 Khai căn nhiều chu trình

Trong kết luận của mục 2.1, ta thấy rằng khi độ dài chu trình và số mũ không nguyên tố cùng nhau, một chu trình sẽ tách thành $\gcd(l, k)$ chu trình. Điều này gợi ý rằng nếu ta ghép các chu trình bị tách ra theo cách tương tự, ta sẽ thu được quá trình ngược lại.

Thật vậy, trong phép khai căn, ta có thể lấy k chu trình cùng độ dài l và ghép xen kẽ chúng thành một chu trình lớn. Chu trình lớn có độ dài kl ; khi lấy lũy thừa bậc k , nó tất yếu tách thành đúng k chu trình độ dài l như ban đầu. Vì vậy cách làm này là đúng.

Chẳng hạn với một hoán vị T gồm hai chu trình độ dài 5, việc khai căn bậc hai có hai cách. Cách thứ nhất là ghép xen kẽ hai chu trình thành một chu trình lớn độ dài 10. Cách thứ hai là áp dụng riêng phương pháp của mục 3.1 cho từng chu trình, khi đó kết quả vẫn là hai chu trình nhỏ. Cả hai cách đều có thể kiểm chứng là đúng.

Có nhất thiết phải chọn đúng k chu trình cùng độ dài để ghép không? Không. Nếu ta chọn m chu trình cùng độ dài l và ghép chúng trong một bước, chỉ cần bảo đảm rằng lũy thừa bậc k của chu trình độ dài ml sẽ tách nó thành m phần. Điều kiện tương ứng là

$$\gcd(ml, k) = m.$$

Từ đó thấy m là một ước của k , đồng thời

$$\gcd\left(l, \frac{k}{m}\right) = 1.$$

Một điều kiện đủ hiển nhiên là m là bội của $\gcd(l, k)$, và giá trị nhỏ nhất như vậy là

$$m = \gcd(l, k).$$

Quay lại mục 3.1: nếu độ dài và số mũ không nguyên tố cùng nhau, một chu trình đơn lẻ không thể được khai căn bằng cách ở mục đó. Tuy nhiên, ta có thể chọn số lượng tương ứng các chu trình cùng độ dài rồi ghép xen kẽ để hoàn thành quá trình khai căn. Nếu trong tình huống này không tìm được đủ số chu trình cùng độ dài, chắc chắn không có nghiệm. Lý do là kết quả của phép lũy thừa nguyên là duy nhất, và ở trên ta đã liệt kê hết mọi khả năng của phép lũy thừa nguyên; do đó không tồn tại hoán vị T' nào có lũy thừa bậc k bằng hoán vị nguồn trong trường hợp ấy.

Gộp hai trường hợp lại, ta có quy trình sau:

$$m = \gcd(l, k).$$

Chọn nm chu trình độ dài l , với n là số nguyên dương và

$$\gcd\left(nm, \frac{k}{m}\right) = 1,$$

để bảo đảm bước tiếp theo khả thi. Ghép xen kẽ các chu trình này thành một chu trình lớn, rồi khai căn bậc k/m của chu trình lớn.

3.3 Hiện thực thuật toán

So với lũy thừa nguyên, lũy thừa phân số, tức khai căn, phức tạp hơn nhiều. Cần chia nhiều trường hợp, và nghiệm cũng không duy nhất. Dưới đây là một thuật toán $\mathcal{O}(n)$ có thể tìm ra một nghiệm đúng:

Sắp xếp các chu trình của hoán vị nguồn theo độ dài chu trình, có thể dùng sắp xếp thùng. Với mỗi chu trình nguồn có độ dài l , đặt $m = \gcd(l, k)$. Nếu còn ít nhất m chu trình cùng độ dài chưa xử lý, ghép xen kẽ m chu trình đó vào mảng đích và lưu chúng thành một chu trình lớn. Sau đó bắt chước thuật toán ở mục 2.3 để khai căn bậc k/m của chu trình lớn và lưu kết quả. Nếu không còn đủ m chu trình cùng độ dài, dừng lại và báo vô nghiệm.

Vì bước sắp xếp dùng sắp xếp thùng, và thao tác trên mỗi chu trình đều là $\mathcal{O}(l)$, trong đó khi ghép có thể dùng m con trỏ, lần lượt dịch xuống và xuất ra phần tử, nên tổng độ phức tạp thời gian là $\mathcal{O}(n)$. Hằng số quá thật hơi lớn, nhưng so với việc không có cách xử lý thì đã tốt hơn nhiều.

3.4 Ví dụ tối ưu: máy xáo bài

Mô tả bài toán. Kai Kai và Fan Fan có N lá bài, được đánh số lần lượt từ 1 đến N , và một máy xáo bài. Giả sử N là số lẻ. Máy xáo bài thực hiện thao tác sau: với mọi vị trí I , $1 \leq I \leq N$, nếu lá bài ở vị trí I là J , và lá bài ở vị trí J là K , thì sau khi đi qua máy xáo bài, vị trí I sẽ chứa lá bài K .

Kai Kai trước hết viết một hoán vị a_i của các số 1 đến N . Ở vị trí a_i , cậu đặt lá bài có số a_{i+1} , nhờ đó thu được thứ tự ban đầu $x_1, x_2, \dots, x_N$. Cậu đưa bộ bài ở thứ tự này vào máy xáo bài S lần, thu được thứ tự cuối cùng $p_1, p_2, \dots, p_N$. Nay Kai Kai nói cho Fan Fan thứ tự cuối cùng và số lần xáo, và Fan Fan phải đoán thứ tự ban đầu $x_1, x_2, \dots, x_N$.

Dữ liệu vào. Dòng đầu gồm hai số nguyên N và S , với $1 \leq N \leq 1000$ và $1 \leq S \leq 1000$. Dòng thứ hai gồm thứ tự cuối cùng của các lá bài $p_1, p_2, \dots, p_N$.

Dữ liệu ra. In ra một dòng, tức thứ tự ban đầu $x_1, x_2, \dots, x_N$.

Phân tích thuật toán. Rõ ràng trong bài này, một bộ bài chính là một hoán vị, và mỗi lần xáo là một phép bình phương hoán vị. Vì số lá bài là số lẻ, và ban đầu các lá bài tạo thành một chu trình lớn, nên khi bình phương sẽ không bị tách. Do đó ở mọi thời điểm, hoán vị biểu diễn thứ tự các lá bài luôn là một chu trình lớn.

Theo định lý ở đầu bài viết, nếu $T^k = e$, với T là một chu trình, thì nghiệm nguyên dương nhỏ nhất của k là độ dài của T . Vì vậy lũy thừa bậc N của chu trình này là chu trình đơn vị. Nói cách khác, nếu

$$k \bmod N = 1,$$

thì lũy thừa bậc k của chu trình bằng chính nó. Mỗi lần xáo là một phép bình phương đơn giản, nên xáo x lần tương ứng với lấy lũy thừa bậc 2^x của chu trình ban đầu.

Vì N là số lẻ, phương trình

$$2^x \bmod N = 1$$

nhất định có một nghiệm nguyên $x < N$; giả sử nghiệm ấy là a . Điều đó có nghĩa là một bộ bài sau khi xáo a lần sẽ quay lại thứ tự ban đầu. Dùng thứ tự cuối cùng và tiếp tục xáo cho đến khi quay về thứ tự này, ta tìm được độ dài chu kỳ a . Sau đó mô phỏng thêm $a - S$ lần là thu được thứ tự ban đầu.

Thuật toán trên là lời giải chuẩn do ban tổ chức đưa ra. Độ phức tạp thời gian của nó là $\mathcal{O}(N^2 + \log S)$.

Nhìn theo hướng khác, sau khi đã biết kết quả và S , ta có thể trực tiếp khai căn theo phương pháp mục 3.1 trong S lần để thu được kết quả, với độ phức tạp $\mathcal{O}(NS)$.

Còn đơn giản hơn, ta tính 2^S , rồi khai căn bậc 2^S trực tiếp trên hoán vị đích. Ở đây có một mẹo nhỏ: khi khai căn, ta chỉ cần biết phải tiến bao nhiêu bước trong chu trình, nên chỉ quan tâm tới

$$2^S \bmod N,$$

tránh được phép tính số lớn. Do đó việc tính 2^S cần $\mathcal{O}(\log S)$, còn khai căn cần $\mathcal{O}(N)$. Tổng độ phức tạp thời gian là $\mathcal{O}(N + \log S)$.

4 Tổng kết

Bài toán lũy thừa trên nhóm hoán vị được nghĩ ra từ ví dụ cuối cùng, bài máy xáo bài. Tất cả bắt nguồn từ việc nghiên cứu vấn đề sâu hơn. Sự tách chu trình là hiện tượng tự nhiên, còn phép ghép chu trình là điều ta chủ động dựng lên; đó là kết quả của cách tư duy đảo ngược. Thông qua tách và ghép, phép

lũy thừa trên nhóm hoán vị được giải quyết trọn vẹn; kết quả ấy lại đến từ việc thử nhiều ví dụ và đưa ra nhiều phỏng đoán.

Mỗi khi phát hiện vấn đề, tìm hiểu vấn đề và giải quyết vấn đề, con người tìm được con đường tiến bộ. Khi hoàn thành quá trình đó, con người đã tiến bộ.

Tài liệu tham khảo

- Liu Rujia, *The Art of Algorithms and Informatics Olympiad*.
- John D. Dixon và Brian Mortimer, *Permutation Groups*, Graduate Texts in Mathematics, số 163.

A Nguyên văn đề bài “Shuffle Machine”

Alice and Bob have a set of N cards labelled with numbers $1, \dots, N$, so that no two cards have the same label, and a shuffle machine. We assume that N is an odd integer.

The shuffle machine accepts the set of cards arranged in an arbitrary order and performs the following double-shuffle operation: for all positions i , $1 \leq i \leq N$, if the card at position i is j , and the card at position j is k , then after the operation, position i will hold card k .

Alice and Bob play a game. Alice first writes down all the numbers from 1 to N in some random order $a_1, a_2, \dots, a_N$. Then she arranges the cards so that position a_i holds the card numbered a_{i+1} for every $1 \leq i \leq N - 1$, while position a_N holds the card numbered a_1 . This produces an order $x_1, x_2, \dots, x_N$, where x_i is the card at the i -th position.

She then performs S double shuffles with the machine. After that, the cards are in a final order $p_1, p_2, \dots, p_N$, which Alice reveals to Bob together with S . Bob's task is to guess the original order $x_1, x_2, \dots, x_N$, just before the cards were given to the machine.

Input. The first line of CARDS.IN contains two integers separated by one blank: the odd integer N , $1 \leq N \leq 1000$, and the number of cards, and the integer S , $1 \leq S \leq 1000$, the number of double-shuffle operations. The following N lines describe the final order after all shuffles; for each i , $1 \leq i \leq N$, the $(i + 1)$ -st line contains p_i , the card at position i .

Output. The output file CARDS.OUT should contain N lines describing the order of cards just before they were given to the shuffle machine. For each i , $1 \leq i \leq N$, the i -th line should contain x_i .